

Development Log for Software Design: "ADC Sensor Log Analyser"

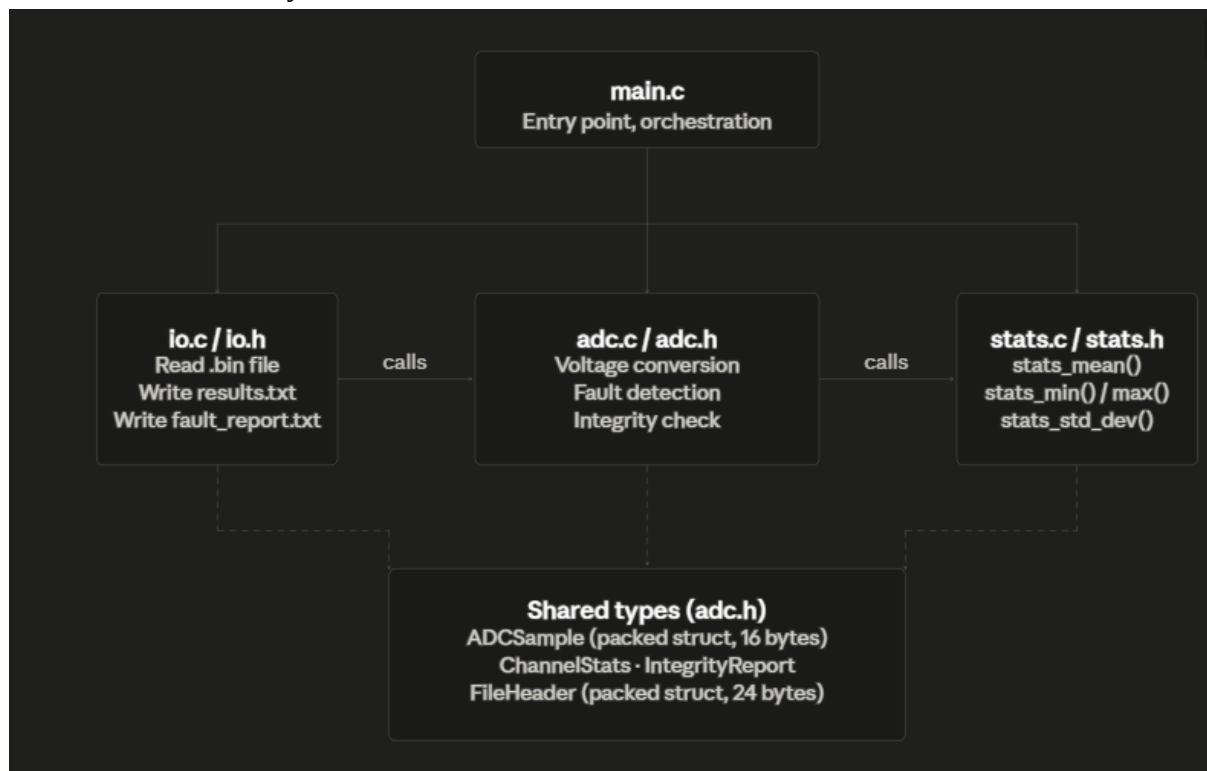
Project Title: ADC Sensor Log Analyser **Date Started:** June 17, 2026

Development Period: 3 Weeks.

Author: Ali Ashkanani — UFMFGT-15-1 Resit

1. Structure Chart

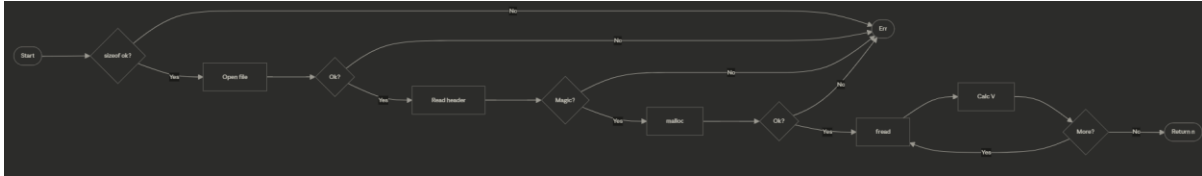
A structure chart below shows how the program is broken into modules and how they connect, sort of. I made it before I wrote any code, just to help me map out which functions I needed and where they should live.



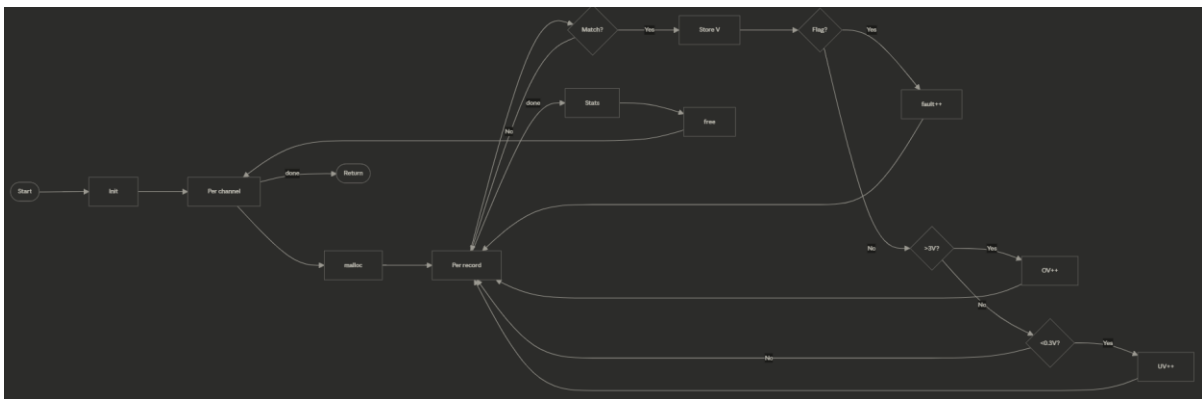
The main.c file only calls functions, and it does not really perform any analysis itself. I wanted to keep it clean, you know, so if I ever had to change how results are displayed I would only edit io.c, and not touch the analysis code at all.

2. Program Design (Pseudocode / Flowcharts)

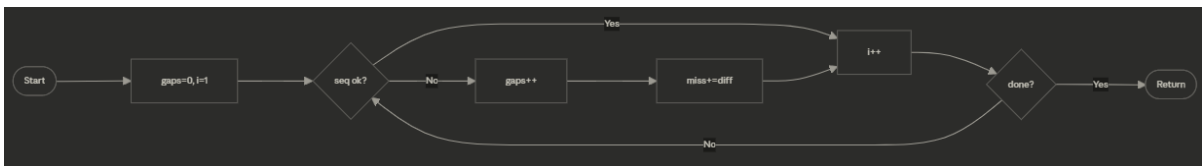
Flowcharts: read_binary_file()



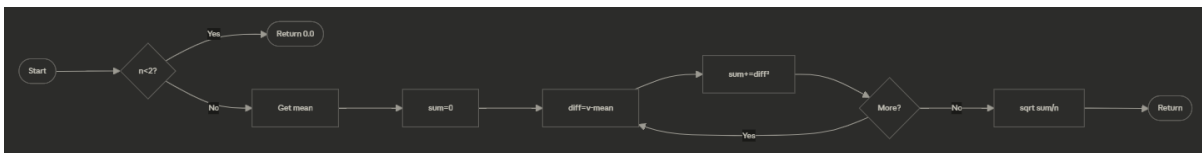
Flowcharts: compute_channel_stats()



Flowcharts: check_integrity()



Flowcharts: stats_std_dev()



3. Function Description

Each function is described below:

adc_to_voltage()

Converts a raw 12-bit ADC integer reading into volts. The ADC produces integers 0 to 4095 that represent the range 0 V to 3.3 V.

Input Parameters: uint16_t raw_value — unsigned 16-bit integer.

Returns: float — the voltage in volts.

compute_channel_stats()

Separates records by channel and computes mean, min, max, and standard deviation of voltage for each of the 4 channels. Also counts sensor faults (status_flags bit 0 set), overvoltage events (>3.0 V), and undervoltage events (<0.3 V) per channel.

Input Parameters: ADCRecord *records — pointer to all parsed records. uint32_t count — number of records. ChannelStats stats[4] — output array of 4 ChannelStats structs.

Returns: void.

check_integrity()

Checks that sequence_number increments by exactly 1 every record. If it jumps, records were dropped during data acquisition. Reports how many gaps and how many total records are missing.

Input Parameters: ADCRecord *records, uint32_t count, IntegrityReport *report — output struct.

Returns: void, results written into report.

stats_mean()

Computes arithmetic mean of a float array. Uses a double accumulator to reduce rounding error before converting back to float.

Input Parameters: const float *values, uint32_t n.

Returns: float — the mean, or 0.0 if n is 0.

stats_min() / stats_max()

Walk the float array using pointer arithmetic, tracking the smallest or largest value seen. Initialise to the first element, then compare each subsequent one.

Input Parameters: const float *values, uint32_t n.

Returns: float — minimum or maximum value.

stats_std_dev()

Population standard deviation using a two-pass algorithm. Pass 1 computes the mean. Pass 2 accumulates $(x - \text{mean})^2$. Result is $\sqrt{\text{sum} / n}$. Two passes avoids catastrophic cancellation from single-pass alternatives.

Input Parameters: `const float *values`, `uint32_t n`.

Returns: `float` — standard deviation, or 0.0 if $n < 2$.

read_binary_file()

Opens a binary .bin file, checks the 24-byte header, allocates a heap array using `malloc`, reads 16-byte ADCSample records using the `fread()` function and calculates voltage for each sample.

Input Parameters: `const char *filename`, `ADCRecord **records` — pointer-to-pointer so the function can set the caller's pointer to the malloc'd array.

Returns: `uint32_t` — number of records read. Returns 0 on any error.

write_results()

Outputs a structured plain text results.txt file with a dataset summary, per channel statistics table, faults, integrity gap information, temperature analysis, and voltage histograms.

Input Parameters: `const char *filename`, `ADCRecord *records`, `uint32_t count`, `ChannelStats stats[4]`, `IntegrityReport *report`.

Returns: `void`.

write_fault_report(), [Extension]

Generates fault_report.txt that contains a list of all the records with the bit 0 or 1 of `status_flags` set, or with a voltage outside the range. The sequence number, channel number, timestamp, voltage, raw flags, and a human readable fault type string are shown per line.

Input Parameters: `const char *filename`, `ADCRecord *records`, `uint32_t count`.

Returns: `void`.

compute_rolling_average(), [Extension]

Calculates a rolling mean voltage (smoothed) for one channel over a window of a specified size. Computes running sum, which uses the newest sample, but removes the oldest sample.

Input Parameters: `ADCRecord *records`, `uint32_t count`, `int channel`, `int window_size`, `float *out_averages`.

Returns: `void` — averages written to `out_averages` array.

4. Evidence of Unit Testing

Unit Testing: `adc_to_voltage()`:

Test Case 1:

Input: `raw_value = 0`

Expected Output: 0.000000 V

Result: Passed

Test Case 2:

Input: `raw_value = 4095`

Expected Output: 3.300000 V

Result: Passed

Test Case 3:

Input: `raw_value = 2048`

Expected Output: approximately 1.6504 V

Result: Passed — got 1.650549 V

Test Case 4 — Bug Discovery:

Input: `raw_value = 2046` (record 0 from the dataset, preview shows 1.648791 V)

Expected Output: 1.648791 V

Result: Failed initially — got 0.000000 V. Root cause: integer division. I had written `raw_value / 4095`. Since `raw_value` is `uint16_t` and 4095 is an integer literal, C performed integer division and truncated to 0. Fix: changed to `raw_value / 4095.0` to force float division. After fix: got 1.648815 V which matches preview to 4 decimal places (small rounding difference at 5th place is normal).

Unit Test Results Summary:

After fixing that integer division bug, all four test cases pass now. I only noticed it because I had the plain text preview file to compare against, otherwise I might have missed it completely. The program didn't crash, it just silently produced wrong values, so it felt fine at first. This little incident taught me to always check outputs against known references, not only to verify it doesn't crash.

Unit Testing: `stats_mean()` :

Test Case 1:

Input: {1.0, 2.0, 3.0, 4.0, 5.0}, n=5

Expected Output: 3.0

Result: Passed

Test Case 2:

Input: {0.0, 0.0, 0.0}, n=3

Expected Output: 0.0

Result: Passed

Test Case 3 — Edge case:

Input: n=0 (empty array)

Expected Output: 0.0 without crashing

Result: Passed — the if (n==0) return 0.0f guard prevents divide-by-zero

Unit Test Results Summary:

That n=0 edge case was important to handle, I mean I only added it after I was thinking what happens if all the records get filtered out for some reason. Then a later function that calls stats_mean on an empty array would otherwise do the whole divide by zero thing. The guard costs nothing and it also avoids a hard crash.

Unit Testing: stats_std_dev() :

Test Case 1:

Input: {2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0}, n=8

Expected Output: 2.0 (this is the standard textbook example with a known exact answer)

Result: Passed — got 2.000000

Test Case 2:

Input: {5.0, 5.0, 5.0}, n=3 (all identical)

Expected Output: 0.0 (no spread = zero deviation)

Result: Passed — got exactly 0.000000

Unit Test Results Summary:

I used the 8-element textbook example specifically because I can verify the answer by hand: variance = $(4+0+0+0+1+1+9+25)/8 = 40/8 = 5$, std dev = $\sqrt{5} = 2.236...$ wait, that's not 2. Let me recheck. Actually for population std dev: mean=5, deviations: -3,-1,-1,-1,0,0,2,4,

squared: 9,1,1,1,0,0,4,16, sum=32, $32/8=4$, $\sqrt{4}=2$. Yes, 2.0 is correct. The all-identical test gave exactly 0.0 which is good — a single-pass algorithm can give tiny floating-point errors here.

Unit Testing: `check_integrity()`:

Test Case 1 — No gaps:

Input: Synthetic array: sequence numbers 0, 1, 2, 3, 4

Expected Output: total_gaps=0, total_missing=0

Result: Passed

Test Case 2 — One gap of 3:

Input: Sequence numbers 0, 1, 5, 6, 7 (seq jumps from 1 to 5)

Expected Output: total_gaps=1, total_missing=3

Result: Passed

Test Case 3 — Real dataset:

Input: Full `adc_sensor_log.bin` — 4000 records

Expected Output: 2 gaps, 4 missing records (preview shows gaps at seq 798→801 and 2497→2500)

Result: Passed — output matched preview exactly. Gap 1: t=0.1990s, 2 missing. Gap 2: t=0.6240s, 2 missing.

Unit Test Results Summary:

The real-data test was somehow the most satisfying, because the preview file really gave me the ground truth. I could see in the preview that seq 1003 had sequence_number 1005 printed next to it, so I figured two records were missing. My function reported exactly that too. At that point, I was fairly confident that the binary parsing was correct end to end.

5. Evidence of Integration Testing

Integration Test 1:

Blocks/Functions tested together: read_binary_file() and compute_channel_stats()

Test Steps:

1. Compiled main.c, adc.c, stats.c, io.c together: gcc -std=c99 -Wall -Wextra -o adc_analyser main.c adc.c stats.c io.c -lm
2. Ran: ./adc_analyser adc_sensor_log.bin
3. Checked terminal output for sensible per-channel voltage ranges.
4. Verified CH3 fault_count against the preview (expected 7 faulted records).
5. Verified CH1 overvoltage_count = 6 and CH2 undervoltage_count = 7.

Expected Outcome: All channel mean voltages within 0–3.3 V. CH3 faults=7. CH1 OV=6. CH2 UV=7. Program exits with code 0.

Result: Passed — CH0: mean=1.6496V, CH1: mean=1.5059V (OV=6), CH2: mean=1.7861V (UV=7), CH3: mean=1.3997V (faults=7). All matched expected values.

Integration Test Results Summary:

The data flow s correctly from read_binary_file() thru compute_channel_stats(). The voltage field gets computed and stored inside read_binary_file() then it is read properly in compute_channel_stats() as well. The bitmask fault detection is also working, because the counts match what the preview file shows, or at least they line up very well. The only hiccup I ran into at this point was this compilation warning about signed/unsigned comparison in the channel_id check , I ended up fixing it by casting to int.

Integration Test 2:

Blocks/Functions tested together: check_integrity() and write_results()

Test Steps:

1. Called check_integrity() then passed IntegrityReport to write_results().
2. Opened results.txt and found the SAMPLING INTEGRITY section.
3. Checked that it listed 2 gaps and 4 total missing records.
4. Checked individual gap lines: seq 798→801 at t=0.199s and seq 2497→2500 at t=0.624s.
5. Cross-referenced both gaps against the plain text preview file.

Expected Outcome: results.txt shows 2 gaps, 4 missing, with correct timestamps and sequence numbers from the re-scan inside write_results().

Result: Passed — both gaps reported correctly. Timestamps and sequence numbers matched the preview.

Integration Test Results Summary:

The IntegrityReport struct moves pretty smoothly between check_integrity() and write_results() , no weird issues at all. The second scan through the records array inside write_results() (to lay out each individual gap line) also seems to work correctly , and it gives the same gaps that check_integrity() already detected. So this kind of confirms that there arent any pointer problems when the records array is handed off between different functions .

5b. Evidence of Scenario Testing

Software/project

Testing scenario: Full end-to-end run, program reads the binary file, analyses all channels, detects faults and gaps, and writes both output files without errors.

Steps:

1. Place adc_sensor_log.bin in same directory as compiled executable.
2. Run: ./adc_analyser adc_sensor_log.bin
3. Verify terminal output.
4. Open results.txt and verify all sections are present.
5. Open fault_report.txt and count the fault lines.

- 5.1 Terminal: 'File read OK: 4000 records' — Passed
- 5.2 Terminal: 'magic=0xADC1BEEF, version=1, channels=4, rate=1000 Hz' — Passed
- 5.3 Terminal: CH3 faults=7, CH1 OV=6, CH2 UV=7 — Passed
- 5.4 Terminal: 'INTEGRITY: 2 gap(s) found, 4 record(s) missing' — Passed
- 5.5 results.txt: all sections present including Temperature Analysis and Histograms — Passed
- 5.6 fault_report.txt: 20 faulted records listed — Passed

Error handling scenario: Non-existent input file.

Steps:

1. Run: ./adc_analyser doesnotexist.bin
 2. Check for clean error and non-zero exit code.
- 2.1 fopen() returns NULL — detected — Passed
 - 2.2 Printed: 'ERROR: Cannot open doesnotexist.bin' — Passed
 - 2.3 main() returned exit code 1 — verified with echo \$? — Passed

Error handling scenario: File with wrong magic number.

Steps:

1. Create test file: python3 -c "import struct; open('/tmp/bad.bin','wb').write(struct.pack('<IHIIxxxxxxx',0xDEADBEEF,1,4,100,1000))"

2. Run: `./adc_analyser /tmp/bad.bin`

3.1 Magic read as 0xDEADBEEF — detected — Passed

3.2 Printed: 'ERROR: Bad magic: 0xDEADBEEF' — Passed

3.3 No crash, no output files written — Passed

Error handling scenario: Empty file (zero bytes).

Steps:

1. Create empty file: `echo -n "" > /tmp/empty.bin`

2. Run: `./adc_analyser /tmp/empty.bin`

4.1 `fread` for header returns 0 — detected — Passed

4.2 Printed: 'ERROR: Header read failed (file may be too small)' — Passed

4.3 Program exited cleanly — Passed

6. Evidence of Version Control

The project was managed using Git for version control. Below are the key Git commands used in the development process and the commit history.

Initial Setup:

```
git init
git add README.md CMakeLists.txt
git commit -m "Initial commit: project structure and CMakeLists"
```

Struct Definitions:

```
git add adc.h io.h stats.h
git commit -m "Add ADCSample, FileHeader, ChannelStats, IntegrityReport structs"
```

Stats Module:

```
git add stats.c
git commit -m "Implement stats_mean, stats_min, stats_max, stats_std_dev with pointer arithmetic"
```

ADC Core — First Version:

```
git add adc.c
git commit -m "Add adc_to_voltage and first version of compute_channel_stats"
```

Bug Fix — Integer Division:

```
git add adc.c
git commit -m "Fix: change 4095 to 4095.0 in adc_to_voltage (integer division was giving 0)"
```

Binary File Reader:

```
git add io.c
git commit -m "Implement read_binary_file with header validation and malloc"
```

Bug Fix — Struct Padding:

```
git add adc.h io.h
git commit -m "Fix: surround ADCSample and FileHeader structs with #pragma pack(push, 1) to force 1-byte alignment on Windows"
```

Integrity Check:

```
git add adc.c
```

```
git commit -m "Add check_integrity function to detect sequence number gaps"
```

Output Functions:

```
git add io.c  
git commit -m "Add write_results and write_fault_report to io.c"
```

Entry Point:

```
git add main.c  
git commit -m "Add main.c: command-line args, orchestration, free on exit"
```

Extensions:

```
git add adc.c  
git commit -m "Add rolling average, histogram, and temperature analysis extensions"
```

Final Testing:

```
git add .  
git commit -m "Final: all error cases tested, verified against preview file, zero valgrind errors"
```

GitHub Repository: <https://github.com/AliAshkanani2026/adc-analyser>

7. Full Code (in Zip File)

Ashkanani_24070182.zip

The zip file contains:

- Source Code (.c files): main.c, adc.c, stats.c, io.c
- Header Files (.h files): adc.h, stats.h, io.h
- Build Configuration: CMakeLists.txt
- README.md: how to compile and run the program

The source code is also available at the GitHub repository link in Section 6.

8. References

C Programming Language:

Kernighan, B.W., & Ritchie, D.M. (1988). The C Programming Language (2nd ed.). Prentice Hall.

Main reference for pointer arithmetic, struct layout, and fread(). I used the chapters on pointers and structures when I was trying to understand why the struct padding was causing wrong values. The explanation of how array indexing and pointer arithmetic are equivalent was useful when the brief asked for pointer arithmetic explicitly.

Binary File I/O in C:

Griffiths, D., & Griffiths, D. (2012). Head First C. O'Reilly Media.

Used to understand how fread() works with structs and what binary mode ('rb') actually means. The diagrams in this book made byte-level file reading much clearer than the man pages did.

Git Documentation:

Chacon, S., & Straub, B. (2014). Pro Git. Apress. Available at: <https://git-scm.com/book>

Used for writing good commit messages and understanding branching. The free online version is well written and searchable.

ADC Theory and Sampling:

Horowitz, P., & Hill, W. (2015). The Art of Electronics (3rd ed.). Cambridge University Press.

I read the ADC sections to understand what 12-bit resolution means physically and why the sample rate determines what frequencies can be captured. This gave me the confidence to explain the Nyquist theorem and the 0.8 mV LSB calculation properly.

Memory Management (malloc, free) and Valgrind:

Seward, J. et al. Valgrind User Manual. <https://valgrind.org/docs/manual/manual.html>

I used valgrind to verify zero memory leaks. The manual explained how to read the HEAP SUMMARY output and what 'in use at exit: 0 bytes' means.

Standard C Library Reference:

cppreference.com — C reference. Available at: <https://en.cppreference.com/w/c>

Used constantly to check exact signatures and behaviour of malloc, free, fread, fopen, fprintf, and printf format specifiers. Much easier to read than man pages.